FinCore — Technical Blueprint & 10-Phase Implementation Roadmap

A Principal Architect's guide to building a production-grade payment processing, double-entry ledger and reconciliation platform with C#, ASP.NET Core, EF Core and SQL Server.

Part 0 — Foundations Before You Write Code

0.1 What FinCore Is

FinCore is the engine that sits behind products like Paymob or Stripe. It accepts payment requests from merchants, talks to external payment providers (PSPs / acquiring banks), records every movement of money in an immutable double-entry ledger, and finally proves that its own books agree with the provider's and the bank's books (reconciliation).

Every design decision in this roadmap serves three promises:

1. **Never lose money** — every accepted payment is durably recorded, even if servers crash mid-request.
2. **Never create money** — every ledger entry balances; debits always equal credits.
3. **Never charge twice** — retries, duplicate webhooks and race conditions cannot move money more than once.

0.2 Domain Glossary (Learn These First)

Term	Plain-language meaning
Merchant	The business that wants to get paid (an online shop). Your customer as a platform.
Customer / Payer	The person paying the merchant.
PSP / Acquirer	Payment Service Provider or bank that actually talks to Visa/Mastercard/Meeza networks. FinCore will integrate with one (simulated).
Authorization	The issuing bank agrees to *hold* the funds. No money has moved yet.
Capture	The merchant claims the held funds. This is the moment revenue becomes real.

Term	Plain-language meaning
Settlement	The PSP/bank actually transfers money (net of fees) to the merchant, usually T+1 to T+3.
Refund	Returning captured money to the customer. Never a deletion — always a new opposite transaction.
Ledger	The book of record of all financial facts, organised in accounts.
Journal Entry	One business event recorded as balanced debit and credit lines.
Idempotency Key	A client-generated token that makes a retried request safe.
Webhook	An asynchronous HTTP callback telling you something happened (payment succeeded, failed).
Outbox	A table that guarantees events are published if and only if the DB transaction committed.
Reconciliation	Comparing two independent records of the same money (ours vs. provider's vs. bank's) and explaining every difference.

0.3 Architecture Decisions (Apply to Every Phase)

Decision	Choice	Why
Style	**Modular monolith**, clean layering	Microservices add distributed-transaction pain you do not need yet; module boundaries keep that option open.
Projects	`FinCore.Domain`, `FinCore.Application`, `FinCore.Infrastructure`, `FinCore.Api`, `FinCore.Workers`, `FinCore.ProviderSimulator`, `*.Tests`	Domain has zero dependencies; rules are testable without a database.
Money	`long` minor units (piasters) + ISO-4217 currency code. **Never** `float` / `double`; avoid `decimal` for storage.	Exact arithmetic; no rounding drift. 250.00 EGP is stored as 25000.
IDs	Internal `Guid` + public prefixed id (`pay_…`, `mer_…`)	Safe to expose; hard to enumerate.

Decision	Choice	Why
Time	`DateTimeOffset` in UTC only	Prevents timezone bugs in settlement cut-offs.
Errors	RFC 7807 `ProblemDetails` with stable `code` field	Clients can branch on codes, not on messages.
Concurrency	`rowversion` optimistic concurrency + DB constraints	The database is the last line of defence; code alone is never trusted.
Versioning	`/api/v1/...`	Financial APIs live for years.

0.4 Roadmap at a Glance

Phase	Name	Core concept
1	Foundation & Payment API	Money value object, aggregates, merchant auth
2	Idempotency & Lifecycle State Machine	Safe retries, legal state transitions
3	Double-Entry Ledger Core	Accounts, journal entries, balance invariant
4	Ledger Immutability, Reversals & Balances	Append-only data, refunds as reversals, concurrency
5	Payment Provider Simulator & Timeouts	Unreliable external systems, unknown outcomes
6	Webhook Ingestion & Outbox Pattern	Signatures, dedup, reliable event publishing
7	Reconciliation Engine & Discrepancy Detection	Three-way matching, exception management
8	Background Workers, Retries & Caching	Resilience, backoff, dead letters, performance
9	Security & Observability	Hardening, audit, metrics, tracing
10	Docker, CI/CD & Production Readiness	Containers, pipelines, load and chaos testing

0.5 The Five Golden Invariants (Memorise Them)

1. **Sum of all debits = sum of all credits**, per journal entry and globally.
2. **Ledger rows are never updated or deleted** — corrections are new rows.
3. **A payment moves only along legal state transitions.**
4. **The same idempotency key + same payload = the same result, exactly once.**
5. **Every external fact (webhook, provider report) is stored raw before it is interpreted.**

Phase 1: Foundation & Payment API

1. **Domain Logic & Conceptual Explanation:**
   - **Who is who.** A *Merchant* is a business receiving money. A *Customer* is the payer. A *Payment* is the merchant's request to collect a specific amount from a specific customer. Payment is the central aggregate: ledger entries, webhooks and reconciliation records will all hang off its identity.
   - **Money is not a floating-point number.** In binary floating point, `0.1 + 0.2 = 0.30000000000000004`. Accounting requires totals that tie out to the last piaster, so we store **minor units** as `long` plus a currency code. Currencies have different exponents (EGP and USD = 2, JPY = 0, KWD = 3), so a `Money` value object owns conversion, and it refuses to add EGP to USD.
   - **Create vs. execute.** Stripe separates *declaring an intention to collect* (PaymentIntent) from *executing* it. Paymob does the same with an order and a payment key. In this phase we only **declare** the payment; nothing touches a provider yet.
   - **Aggregates enforce their own rules.** A `Payment` cannot exist with a zero or negative amount, an unsupported currency, or without a merchant. These rules live in the domain constructor, not in controllers, so they hold no matter who calls the code.
   - **Real-world problem solved.** Without a single, trustworthy payment record, support cannot answer: *"What happened to my 500 EGP?"* Everything in FinCore starts from that record.
2. **Phase Inputs (Prerequisites):**
   - .NET 8 SDK, SQL Server (local Docker container is fine), EF Core 8, FluentValidation, xUnit.
   - The architecture decisions in Part 0 and an empty solution with the project layout from section 0.3.
   - A seed list of supported currencies: `EGP`, `USD`, `EUR`, `SAR`, with their exponents.
3. **Endpoints & Component Deliverables:**
   Endpoints

Method	Route	Auth	Purpose
POST	`/api/v1/merchants`	Admin key	Onboard a merchant; returns the API key **once**.
POST	`/api/v1/customers`	Merchant key	Create a customer under the calling merchant.
POST	`/api/v1/payments`	Merchant key	Create a payment in `Created` status.
GET	`/api/v1/payments/{id}`	Merchant key	Retrieve one payment (only the owner merchant).
GET	`/api/v1/payments`	Merchant key	List with filters `status`, `from`, `to`, `limit`, `cursor`.

POST /api/v1/payments

```
// Request (header: X-Api-Key: fc_live_…)
{ "amount": 25000, "currency": "EGP", "customerId": "cus_8H2K…",
"description": "Order #1042", "metadata": { "orderId": "1042" } }

// 201 Created
{ "id": "pay_01J9Z…", "status": "created", "amount": 25000, "currency":
"EGP", "customerId": "cus_8H2K…", "createdAt": "2026-10-14T09:30:00Z" }

// 422 Unprocessable Entity (ProblemDetails)
{ "type": "https://fincore.dev/errors/validation", "title": "Validation
failed", "status": 422, "code": "amount_invalid", "errors": { "amount":
["Must be greater than 0"] } }
```

POST /api/v1/merchants returns `{ "id": "mer_…", "name": "…", "apiKey": "fc_live_…" }`. Only the SHA-256 hash and an 8-character prefix are stored; the raw key is never retrievable.

Components

- `ExceptionHandlingMiddleware` → maps domain exceptions to ProblemDetails; never leaks stack traces.
- `CorrelationIdMiddleware` → reads or generates `X-Correlation-Id`, stores it in log scope, echoes it in responses.
- `ApiKeyAuthenticationHandler` → hashes the incoming key, resolves the merchant, rejects suspended merchants.
- FluentValidation filter for request DTOs; Swagger/OpenAPI with API-key security scheme.

- EF Core Fluent configurations, a value converter for `Money`, and the first migration.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables**
     - `Merchants` (Id, PublicId, Name, Email, ApiKeyHash, ApiKeyPrefix, Status, CreatedAt)
     - `Customers` (Id, PublicId, MerchantId FK, Email, Name, ExternalRef, CreatedAt) with unique index `(MerchantId, ExternalRef)`
     - `Payments` (Id, PublicId unique, MerchantId FK, CustomerId FK, AmountMinor `bigint`, Currency `char(3)`, Status `varchar(32)`, Description, MetadataJson, CreatedAt, UpdatedAt, RowVersion)
   - **Constraints:** `CHECK (AmountMinor > 0)`, `CHECK (Currency IN (...))`, indexes `(MerchantId, CreatedAt DESC)` and `(MerchantId, Status)`.
   - Working solution skeleton, Swagger UI, migration scripts and a unit-test project covering `Money`.
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] `Money(100, EGP) + Money(50, USD)` throws a domain exception; `Money(100, EGP) + Money(50, EGP)` equals 150.
   - [ ] A payment with amount `0`, `-5`, or `9223372036854775808` (overflow) returns 422 and writes nothing.
   - [ ] An unsupported currency (`XYZ`) returns 422 with code `currency_unsupported`.
   - [ ] Sending `amount: 10.5` (a decimal) is rejected; the API only accepts integers.
   - [ ] Merchant A requesting Merchant B's payment gets **404**, not 403 (no existence leak).
   - [ ] Missing, malformed, revoked or suspended-merchant API keys return 401 with ProblemDetails.
   - [ ] The database rejects a direct `INSERT` with `AmountMinor = 0` (constraint fires even bypassing the API).
   - [ ] The raw API key does not exist anywhere in the database or logs.
   - [ ] Cursor pagination returns stable, non-overlapping pages while new payments are being inserted.
   - [ ] Every response carries `X-Correlation-Id`, and the same id appears in the log lines for that request.
   - [ ] Domain unit tests run without a database connection and achieve at least 90% coverage on `Money` and `Payment`.

Phase 2: Idempotency & Lifecycle State Machine

1. **Domain Logic & Conceptual Explanation:**
   - **The double-charge problem.** A mobile user taps *Pay*, the network drops after the server processed the request but before the response arrived, and the client retries. Without protection the customer is charged twice. Networks are unreliable, so **retries are inevitable** and the server must make them safe.
   - **Idempotency keys.** The client sends a unique `Idempotency-Key` header (a UUID). The server stores the key with a **fingerprint** (hash) of the request and the response it produced. A retry with the same key and same payload returns the stored response without re-executing. The same key with a *different* payload is a client bug and is rejected. This is exactly how Stripe works.
   - **Concurrency twist.** Two identical requests can arrive *simultaneously*. Checking "does the key exist?" and then inserting is a race. The solution is **insert-first**: attempt to insert the key row guarded by a unique index; the loser of the race sees a conflict and responds `409` (in progress) or replays the stored result.
   - **Lifecycle state machine.** Real payments move through well-defined stages, and many movements are illegal (you cannot capture a failed payment or refund an uncaptured one). A state machine makes illegal transitions *impossible by construction*:

From	Allowed next states
`Created`	`Processing`, `Cancelled`, `Expired`
`Processing`	`Authorized`, `Failed`, `Pending Review`
`Authorized`	`Captured`, `Cancelled` (void), `Expired`
`Captured`	`PartiallyRefunded`, `Refunded`, `Settled`
`PartiallyRefunded`	`PartiallyRefunded`, `Refunded`, `Settled`
`Failed`, `Cancelled`, `Expired`, `Refunded`, `Settled`	*terminal — no exit*

   - **Why history matters.** Disputes and audits require knowing *when* and *why* a payment changed state, so every transition appends a row to a history table; the current status is merely the latest fact.
2. **Phase Inputs (Prerequisites):**

- Phase 1 output: `Payments`, `Merchants`, `Customers` tables and the authenticated API.
- The transition table above, agreed and documented.
- Decision: key retention window (24 hours) and request-fingerprint algorithm (SHA-256 over canonicalised JSON body + method + route).

3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/payments`	Now requires header `Idempotency-Key`.
POST	`/api/v1/payments/{id}/confirm`	`Created` → `Processing` (provider call comes in Phase 5).
POST	`/api/v1/payments/{id}/capture`	Body `{ "amount": 25000 }`; `Authorized` → `Captured`. Amount must be ⩽ authorized.
POST	`/api/v1/payments/{id}/cancel`	Void before capture.
GET	`/api/v1/payments/{id}/history`	Ordered list of transitions with reason and actor.

Behaviour contract for idempotent endpoints

Situation	Response
First request with a new key	Process normally, store response, return it.
Replay: same key, same body, completed	Return stored status + body, header `Idempotent-Replayed: true`.
Same key, request still running	`409 Conflict`, code `idempotency_in_progress`.
Same key, different body	`422`, code `idempotency_key_reuse`.
Missing key on a mutating endpoint	`400`, code `idempotency_key_required`.
Illegal transition	`409`, code `invalid_state_transition`, body lists current state and allowed next states.

Components

- `IdempotencyFilter` (action filter or middleware) wrapping the handler in a single DB transaction.
- `PaymentStateMachine` in the Domain layer: `Payment.Transition(newStatus, reason)` is the **only** way to change status.

- Optimistic concurrency via `RowVersion`; `DbUpdateConcurrencyException` is mapped to `409`.
- `IdempotencyCleanupService` (hosted service) deleting expired keys; fully hardened in Phase 8.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables**
     - `IdempotencyKeys` (MerchantId, Key, RequestHash, Status `InProgress|Completed`, ResponseStatusCode, ResponseBody, CreatedAt, ExpiresAt) with **unique index `(MerchantId, Key)`**.
     - `PaymentStatusHistory` (Id, PaymentId, FromStatus, ToStatus, Reason, Actor, CorrelationId, OccurredAt) — insert-only.
   - `PaymentStateMachine` with an exhaustive unit-test matrix (every from/to pair).
   - Integration test suite using a real SQL Server container (Testcontainers).
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] Sending the same `POST /payments` twice with the same key creates **exactly one** payment row; both responses are byte-identical.
   - [ ] **Race test:** fire 50 parallel identical requests; exactly one payment is created, one request does the work, the rest get replay or `409`.
   - [ ] Same key with an altered amount returns 422 `idempotency_key_reuse` and creates nothing.
   - [ ] The same key used by two different merchants does not collide.
   - [ ] If the handler throws, the transaction rolls back and the key can be retried (no "poisoned" key left in `InProgress` forever).
   - [ ] The server is killed mid-request; after restart the stale `InProgress` key (older than the lock timeout) can be safely retried.
   - [ ] Every one of the illegal state pairs in the matrix returns 409 and leaves status and history unchanged; every legal pair succeeds.
   - [ ] Two concurrent `capture` calls on the same payment: one succeeds, one gets 409 (rowversion check).
   - [ ] Capturing more than the authorised amount returns 422 `capture_exceeds_authorized`.
   - [ ] Direct SQL `UPDATE` of `Payments.Status` to an illegal value is blocked by a `CHECK` constraint on allowed status names.
   - [ ] `PaymentStatusHistory` has exactly one row per transition, in chronological order, with the correlation id.
   - [ ] Keys older than 24 hours are purged and a reused expired key is treated as new.

Phase 3: Double-Entry Ledger Core

1. **Domain Logic & Conceptual Explanation:**
   - **What double-entry means.** Every financial event has *two sides*: where money came from and where it went. Each event is recorded as a **journal entry** made of **lines**; each line either **debits** or **credits** an account. The golden rule: **total debits = total credits**. If they do not match, the entry is rejected. This is how accountants have caught errors for 500 years, and it is how Stripe, Paymob and every bank detect "money created from nothing".
   - **Debit and credit are not "minus and plus".** They are *directions*, and what they do depends on the account type:

Account type	Increases with	Example in FinCore
Asset (what we hold or are owed)	Debit	Provider Clearing, Bank Account
Liability (what we owe others)	Credit	Merchant Payable
Revenue (what we earn)	Credit	Fee Revenue
Expense (what we spend)	Debit	Provider Fees
Equity	Credit	Retained earnings

   - **The mindset shift for payment platforms.** The money that customers pay does **not** belong to FinCore; it belongs to the merchant. For us it is a **liability** (we owe it to the merchant). Only our fee is revenue. Software engineers who miss this build systems that treat customer money as company income.
   - **FinCore's chart of accounts (starter set)**

Code	Account	Type	Scope
1000	Provider Clearing	Asset	Per currency; money the PSP owes us
1100	Bank Settlement	Asset	Per currency; real cash in our bank
2000	Merchant Payable	Liability	**One per merchant per currency**
4000	Fee Revenue	Revenue	Per currency
5000	Provider Fees	Expense	Per currency

   - **Worked example.** Customer pays 250.00 EGP (25,000 piasters). Our merchant fee is 2.5% = 625. Capture entry:

Account	Debit	Credit
1000 Provider Clearing	25,000	
2000 Merchant Payable (Shop A)		24,375
4000 Fee Revenue		625
Total	**25,000**	**25,000**

Later, when the PSP settles net of its 1.5% fee (375): Dr Bank 24,625, Dr Provider Fees 375, Cr Provider Clearing 25,000. When we pay the merchant: Dr Merchant Payable 24,375, Cr Bank 24,375. Follow the money through the accounts — that is the whole trick.

- **Real-world problem solved.** A plain `Balance` column that you `UPDATE` cannot answer *why* a balance is what it is, cannot be audited, and silently corrupts under bugs. A ledger *is* the explanation.

2. **Phase Inputs (Prerequisites):**
   - Phase 2: state machine, `Captured` transition, idempotent endpoints.
   - Fee policy decision (percentage + fixed fee, rounding rule: **round half up, remainder goes to fee revenue**, so the entry always balances).
   - The chart of accounts above, with merchant payable accounts auto-created on merchant onboarding.
3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/admin/ledger/accounts`	Create a system account (admin only).
GET	`/api/v1/admin/ledger/accounts`	List accounts with balances.
GET	`/api/v1/admin/ledger/accounts/{id}/entries`	Paged statement of lines for one account.
GET	`/api/v1/admin/ledger/journal-entries/{id}`	One entry with all lines.
GET	`/api/v1/admin/ledger/trial-balance?asOf=`	Totals per account; grand total debits must equal credits.

Method	Route	Purpose
POST	`/api/v1/payments/{id}/capture`	Now **posts the capture journal entry in the same DB transaction** as the status change.

```
// GET /api/v1/admin/ledger/journal-entries/je_01K…
{ "id": "je_01K…", "type": "payment_capture", "source": { "type":
"payment", "id": "pay_01J9Z…" }, "currency": "EGP", "postedAt": "2026-10-
14T09:31:10Z",
  "lines": [ { "account": "1000", "side": "debit", "amount": 25000 }, {
"account": "2000:mer_8F…", "side": "credit", "amount": 24375 }, {
"account": "4000", "side": "credit", "amount": 625 } ] }
```

Components

- `LedgerService.PostAsync(JournalEntryDraft)` — the **only** door into the ledger; validates then inserts header + lines atomically.
- `FeeCalculator` (pure function, heavily unit-tested).
- `ChartOfAccountsSeeder` and a merchant-onboarding hook that creates the merchant's payable account.
- Posting invariants enforced in the domain: at least 2 lines; every line amount > 0 (direction is the `Side`, never a negative number); all lines in one currency; all accounts exist, are active and match the entry currency; sum(debit) = sum(credit).
- **Database defence in depth:** header columns `TotalDebit` / `TotalCredit` with `CHECK (TotalDebit = TotalCredit)`; a trigger on `JournalLines` verifying line sums against the header; a unique index on `(SourceType, SourceId, EntryType)` so the same business event can never be posted twice. *If EF Core splits a posting into several statements and the trigger misfires, switch posting to a stored procedure with a table-valued parameter.*

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables**
     - `LedgerAccounts` (Id, Code, Name, Type, NormalSide, Currency, MerchantId nullable, Status, CreatedAt) — unique `(Code, Currency, MerchantId)`.
     - `JournalEntries` (Id, EntryType, SourceType, SourceId, Currency, TotalDebit, TotalCredit, Description, PostedAt, CorrelationId, ReversesEntryId nullable).
     - `JournalLines` (Id, JournalEntryId FK, LineNo, AccountId FK, Side `char(1)` in `D` / `C`, AmountMinor `bigint` with `CHECK > 0`).
   - Working capture → ledger posting with a trial balance report that always nets to zero.

- A documented chart of accounts and a "how to read a journal entry" page in the repo wiki.

5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] Capturing a 25,000-piaster payment produces exactly one entry with lines 25,000 / 24,375 / 625 and the payment becomes `Captured` in the **same transaction**.
   - [ ] An unbalanced draft (debits 100, credits 99) is rejected by the domain **and** by a raw SQL insert attempt.
   - [ ] A single-line entry, a zero-amount line, a negative-amount line and a mixed-currency entry are all rejected.
   - [ ] Posting to a closed or non-existent account is rejected.
   - [ ] Fee rounding: for amounts 1, 33, 99, 101 and 99,999 the entry always balances and fee + merchant share equals the gross amount.
   - [ ] If the ledger insert fails (simulate by throwing), the payment stays `Authorized`; no half-completed state exists.
   - [ ] Capturing the same payment twice (including via parallel requests) produces exactly **one** journal entry (unique index on source).
   - [ ] Trial balance over 1,000 randomly generated payments: total debits = total credits.
   - [ ] Property-based test (FsCheck): for any random valid event sequence, the ledger-wide sum of signed amounts is zero.
   - [ ] Merchant payable balance (credits minus debits) equals the sum of that merchant's captured net amounts.

Phase 4: Ledger Immutability, Reversals & Balances

1. **Domain Logic & Conceptual Explanation:**
   - **Why ledgers are append-only.** Auditors, regulators (central bank rules, PCI/ISO controls) and dispute teams require that history cannot be rewritten. If a developer, DBA or attacker can edit yesterday's entry, no number in the system can be trusted. So: **no UPDATE, no DELETE, ever.** A mistake is fixed with a **reversing entry** (the exact mirror image) followed by a correct entry. The wrong one stays visible, with its correction beside it — just like a pen-and-paper accountant crossing out with a single line and initialling.
   - **Refunds are reversals, not deletions.** Refunding 100.00 EGP of a captured 250.00 EGP payment creates a *new* journal entry: Dr Merchant Payable 10,000, Cr Provider Clearing 10,000. Policy decision documented here: **our fee is non-refundable by default** (configurable), so only the gross refund moves.
   - **Tamper evidence with a hash chain.** Each entry stores `PrevHash` and `EntryHash = SHA256(PrevHash + canonical entry content)`. Changing any historical row breaks every hash after it, making silent tampering detectable — the same principle

as a blockchain, without the buzzwords. (SQL Server 2022 *Ledger tables* are an optional, managed alternative worth comparing in your thesis.)

- **Balances: derived vs. stored.** The truth is the sum of lines; but summing millions of lines per request is slow. So we keep a **projection** (`AccountBalances`) updated in the same transaction as each posting, and a verifier proves the projection equals the sum of lines. Rule: *the projection can be rebuilt from the ledger, never the other way round.*
- **Pending vs. available.** Captured money is **pending** until the provider settles it (T+1…T+3); only settled money is **available** for payout. Merchants see both numbers, and refunds usually draw on available balance.
- **Concurrency is the enemy.** Two refunds hitting the same account simultaneously can both read the same balance and both approve. Updates must be atomic (`UPDATE ... SET Balance = Balance + @delta`) or guarded with `UPDLOCK` /row versions.

2. **Phase Inputs (Prerequisites):**
   - Phase 3 ledger tables and `LedgerService` as the single posting gateway.
   - Decisions: refund policy (fee refundable or not), negative-balance policy (default: **reject** refunds exceeding available balance), hash algorithm (SHA-256).
   - A dedicated SQL login for the application with permissions defined below.
3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/payments/{id}/refunds`	Create a (partial) refund. Requires `Idempotency-Key` .
GET	`/api/v1/payments/{id}/refunds`	List refunds of a payment.
GET	`/api/v1/merchants/me/balance`	Current `pending` and `available` per currency.
POST	`/api/v1/admin/ledger/entries/{id}/reverse`	Admin reversal with mandatory reason.
GET	`/api/v1/admin/ledger/integrity`	Runs hash-chain verification and projection-vs-ledger checks.

```
// POST /api/v1/payments/pay_01J9Z…/refunds   (Idempotency-Key: 7c1f…)
{ "amount": 10000, "reason": "customer_request" }
// 201
{ "id": "ref_01M…", "paymentId": "pay_01J9Z…", "amount": 10000, "status":
"succeeded", "paymentStatus": "partially_refunded" }
// 422 when total refunds would exceed captured amount
{ "status": 422, "code": "refund_exceeds_captured", "title": "Refund
exceeds remaining refundable amount", "remaining": 15000 }
```

Components

- `RefundService`: locks the payment row, computes `remaining = captured - sum(refunds)`, posts the reversal entry, inserts the `Refunds` row, transitions the payment — **one transaction**.
- `LedgerService.ReverseAsync(entryId, reason)`: generates mirrored lines and sets `ReversesEntryId`.
- `BalanceProjector` (inside the posting transaction) using atomic updates.
- `LedgerIntegrityVerifier` (service + endpoint; scheduled in Phase 8).
- **DB permissions:** the app login has `SELECT, INSERT` on ledger tables; `UPDATE, DELETE` are `DENY`. Additionally `INSTEAD OF UPDATE, DELETE` triggers `THROW` so even a privileged mistake is blocked.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables:** `AccountBalances` (AccountId PK, BalanceMinor, LastEntrySequence, RowVersion, UpdatedAt); `Refunds` (Id, PaymentId, AmountMinor, Status, Reason, JournalEntryId, CreatedAt).
   - **Columns added to `JournalEntries`:** `Sequence` (monotonic `bigint` identity), `PrevHash`, `EntryHash`; unique index on `ReversesEntryId` (an entry can be reversed once).
   - SQL scripts for the permission model and anti-tamper triggers.
   - An integrity report format (JSON) listing any broken links or mismatched balances.
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] `UPDATE JournalLines ...` and `DELETE FROM JournalEntries ...` fail for the application login **and** for a `db_owner` session via the triggers.
   - [ ] Refunding 100.00 of a 250.00 payment moves exactly 10,000 piasters, sets status `PartiallyRefunded`, and the trial balance still nets to zero.
   - [ ] Refunding the remaining 150.00 sets `Refunded`; a further refund of 1 piaster returns 422.
   - [ ] **Concurrency test:** 100 parallel refund requests of 5.00 EGP against a 250.00 payment: at most 50 succeed; total refunded never exceeds the captured amount; ledger and `Refunds` table agree.

- [ ] Concurrent refunds against a merchant with insufficient available balance: none drive `Merchant Payable` into forbidden negative territory.
- [ ] Reversing an entry twice returns 409; the reversal's lines are the exact mirror of the original.
- [ ] Manually altering one historical row (done in a test DB with triggers disabled) makes `/integrity` report the **first broken sequence number**.
- [ ] Deleting `AccountBalances` and rebuilding it from lines yields identical numbers (projection is disposable).
- [ ] After 10,000 random operations, `AccountBalances.BalanceMinor` equals the sum of lines for every account.
- [ ] The sum of `pending + available` for a merchant equals its Merchant Payable ledger balance at all times.
- [ ] Refund retried with the same idempotency key creates only one `Refunds` row and one journal entry.

Phase 5: Payment Provider Simulator & Timeouts

1. **Domain Logic & Conceptual Explanation:**
   - **FinCore does not move money; providers do.** In production you integrate with an acquirer or PSP (a bank, a card network gateway, or a local wallet rail). That system is *outside your control*: it is slow sometimes, down sometimes, and occasionally lies by omission. Building a **Provider Simulator** lets you rehearse every nasty scenario deterministically, which is far cheaper than discovering them in production with real money.
   - **Card data never touches FinCore.** Real systems avoid PCI-DSS scope by using **tokens** (`tok_…`) created by the provider's hosted fields. Our simulator accepts magic tokens such as `tok_success` , `tok_decline_insufficient_funds` , `tok_timeout` , `tok_error_500` , `tok_slow_5s` , and `tok_success_no_response` (the provider approves but the response is dropped).
   - **The hardest problem in payments: the unknown outcome.** You send an authorization request and your HTTP call times out. Did the money get held or not? *You do not know.* If you mark the payment `Failed` and the provider actually approved, the customer is charged for nothing; if you retry blindly, you may charge twice. The correct behaviour is:
     a. Never conclude *failure* from a timeout.
     b. Move the payment to `PendingReview` (outcome unknown).
     c. Resolve it by **asking the provider** (inquiry by our reference) or by receiving its webhook (Phase 6).

   d. Always send our own `reference` and an `Idempotency-Key` to the provider so a retry cannot create a second authorization.

- **Never hold a DB transaction open during a network call.** Pattern: *Transaction 1* — record the attempt and set `Processing` (commit). *Network call. Transaction 2* — record the outcome and transition state (commit). If the process dies between them, a sweeper (Phase 8) finds attempts stuck in `Processing` and runs an inquiry.
- **Anti-corruption layer.** The rest of the system talks to `IPaymentProviderClient`, never to the provider's raw DTOs. Provider decline codes are mapped to FinCore's own vocabulary and a flag saying whether retrying can ever help.

Provider code	FinCore code	Retryable?
`51 insufficient_funds`	`card_declined_insufficient_funds`	No (customer must act)
`05 do_not_honor`	`card_declined_generic`	No
`91 issuer_unavailable`	`provider_unavailable`	Yes, later
HTTP 5xx / timeout	`provider_outcome_unknown`	**Inquire first**, then decide

2. **Phase Inputs (Prerequisites):**
   - Phase 2 state machine including `Processing → Authorized | Failed | PendingReview`.
   - Phase 3–4 ledger (capture and refund must call the provider *and* post ledger entries consistently).
   - Decisions: call timeout (5 s), maximum automatic retries for safe operations (2), circuit-breaker thresholds.
3. **Endpoints & Component Deliverables:**
   FinCore API changes

Method	Route	Purpose
POST	`/api/v1/payments/{id}/confirm`	Body `{ "paymentMethodToken": "tok_success" }`; calls the provider; returns `authorized`, `failed` or `pending_review`.
POST	`/api/v1/admin/payments/{id}/sync`	Forces an inquiry to the provider and applies the result.

Method	Route	Purpose
GET	`/api/v1/payments/{id}/attempts`	Lists provider attempts with outcomes.

Provider Simulator (separate ASP.NET project, own port)

Method	Route	Purpose
POST	`/sim/v1/authorizations`	Header `Idempotency-Key`; body `{ amount, currency, token, reference }`.
POST	`/sim/v1/authorizations/{id}/capture`	Capture full or partial amount.
POST	`/sim/v1/authorizations/{id}/void`	Release the hold.
POST	`/sim/v1/refunds`	Refund a captured authorization.
GET	`/sim/v1/authorizations?reference=`	**Inquiry** by merchant reference.
PUT	`/sim/v1/admin/scenarios`	Configure chaos: latency, error rate, duplicate-webhook rate, out-of-order delivery.
GET	`/sim/v1/settlement-reports?date=`	Daily settlement file (used by Phase 7).

```
// Simulator -> approved
{ "id": "sim_auth_77", "status": "approved", "amount": 25000, "currency":
"EGP", "reference": "pay_01J9Z…", "authCode": "A1B2C3" }
// Simulator -> declined
{ "id": "sim_auth_78", "status": "declined", "declineCode": "51",
"message": "Insufficient funds" }
```

Components

- `IPaymentProviderClient` + `SimulatorProviderClient` using `HttpClientFactory`.
- Polly resilience pipeline: per-call **timeout** (5 s), **retry with exponential backoff and jitter** only for idempotent calls, **circuit breaker** that fails fast when the provider is clearly down.
- `PaymentAttempts` and `ProviderCallLogs` recorders (payloads masked, latency measured).
- `PaymentOrchestrator` implementing the two-transaction pattern.

4. **Phase Outputs (Tangible Artifacts):**

- **Tables:** `PaymentAttempts` (Id, PaymentId, AttemptNo, ProviderRef, Status, FinCoreErrorCode, ProviderCode, RequestedAt, CompletedAt) with unique `(PaymentId, AttemptNo)`; `ProviderCallLogs` (Id, AttemptId, Operation, RequestMasked, ResponseMasked, HttpStatus, LatencyMs, CreatedAt).
- A runnable simulator with a documented scenario catalogue (token → behaviour).
- Sequence diagram of the happy path, the decline path and the unknown-outcome path.

5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] `tok_success` leads to `Authorized`; `tok_decline_insufficient_funds` leads to `Failed` with the mapped FinCore code.
   - [ ] `tok_timeout` (provider hangs 30 s) returns to the client within about 5 s with `pending_review`, **never** `failed`.
   - [ ] `tok_success_no_response`: provider holds funds, our call fails; running `/sync` discovers the authorization via inquiry and moves the payment to `Authorized` without creating a second authorization.
   - [ ] A retry of a failed call reuses the same provider idempotency key; the simulator's authorization count for that reference stays 1.
   - [ ] No database transaction is open during the HTTP call (verified with a test that holds the provider and checks `sys.dm_tran_active_transactions`).
   - [ ] Kill the process after the provider approved but before Transaction 2 commits; on restart the payment is still `Processing` with an open attempt that the sync can resolve.
   - [ ] After N consecutive 5xx responses the circuit opens and requests fail in under 50 ms; after the cool-down a probe closes it.
   - [ ] Card-like numbers (13–19 digit strings) are never persisted in `ProviderCallLogs`, application logs or exceptions.
   - [ ] Capture and refund against the provider failing mid-way never leave the ledger posted without a provider-confirmed action (or are clearly marked and later reconciled).
   - [ ] 200 concurrent confirm calls on 200 different payments with 20% injected errors finish with every payment in exactly one legal state.

Phase 6: Webhook Ingestion & Outbox Pattern

1. **Domain Logic & Conceptual Explanation:**
   - **Why webhooks exist.** Many payment outcomes arrive *later*: a 3-D Secure challenge completes, a bank confirms after hours, a wallet payment is approved on the user's phone. The provider cannot wait on an open HTTP call, so it **calls you back** with an

event. This is Stripe's `payment_intent.succeeded` or Paymob's transaction callback.

- **Inbound webhooks are hostile by nature.** Assume they are:
  - **Forged** — anyone can POST to a public URL → verify an **HMAC-SHA256 signature** over the *raw* body with a shared secret, compare in constant time, and reject timestamps older than 5 minutes (replay protection).
  - **Duplicated** — delivery is *at-least-once* → store `(Provider, ExternalEventId)` under a unique index.
  - **Out of order** — `captured` may arrive before `authorized` → compare event time/version and let the state machine decide; stale events are recorded but ignored.
  - **Slow to process** → persist the raw event first, answer `200` immediately, process asynchronously. Rule 5 of the golden invariants: *store before you interpret.*
- **The dual-write problem.** When a payment becomes `Captured` you must (a) update the database and (b) notify the merchant. If you do both separately, one can succeed while the other fails: the DB commits but the process dies before sending (merchant never learns), or the message is sent but the DB rolls back (merchant told about money that does not exist).
- **The Outbox Pattern.** Write the event to an `OutboxMessages` table **in the same transaction** as the state change. A separate dispatcher reads unsent rows and delivers them, marking them done afterwards. Delivery becomes **at-least-once**, so consumers must be idempotent (events carry a unique id). Atomic by DB transaction, reliable by retry.
- **Outbound merchant webhooks** reuse the same machinery: every delivery is signed with the merchant's secret, retried with exponential backoff, and every attempt is logged so support can answer *"did you notify us?"*.

2. **Phase Inputs (Prerequisites):**
   - Phase 5 simulator able to emit signed webhooks (with duplicate/out-of-order options enabled by the scenarios endpoint).
   - State machine and ledger posting reachable from a handler (the webhook processor reuses the same application services as the API).
   - Shared secret configuration (user-secrets locally, vault later).
3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/webhooks/providers/simulator`	Public inbound endpoint. `401` bad signature, `400` stale timestamp, `200` accepted or duplicate.
POST	`/api/v1/webhook-endpoints`	Merchant registers `{ url, events: ["payment.captured", "payment.failed", "refund.succeeded"] }`; secret returned once.
GET	`/api/v1/webhook-endpoints`	List endpoints.
DELETE	`/api/v1/webhook-endpoints/{id}`	Disable an endpoint.
POST	`/api/v1/webhook-endpoints/{id}/rotate-secret`	Rotate secret (old one valid 24 h).
GET	`/api/v1/webhook-endpoints/{id}/deliveries`	Attempt history with status codes and latency.
POST	`/api/v1/webhook-deliveries/{id}/redeliver`	Manual replay.
GET	`/api/v1/events?after=`	Polling fallback listing business events.

```
// Inbound (from provider) - headers: X-Sim-Signature:
t=1760434200,v1=9f2c…
{ "eventId": "evt_sim_5521", "type": "authorization.captured",
"createdAt": "2026-10-14T09:35:00Z", "data": { "id": "sim_auth_77",
"reference": "pay_01J9Z…", "amount": 25000 } }

// Outbound (to merchant) - header: FinCore-Signature: t=…,v1=…
{ "id": "evt_01P…", "type": "payment.captured", "createdAt": "2026-10-
14T09:35:02Z", "data": { "paymentId": "pay_01J9Z…", "amount": 25000,
"currency": "EGP" } }
```

Components

- `WebhookSignatureFilter` reading the raw request body (`EnableBuffering`) before model binding.
- `InboundWebhookProcessor` (`BackgroundService`) claiming rows with `WITH (UPDLOCK, READPAST, ROWLOCK)` so multiple workers never take the same event.

- `OutboxDispatcher` and `MerchantWebhookDeliveryWorker` with backoff schedule 10 s, 1 m, 5 m, 30 m, 2 h, 12 h, then dead-letter.
- **SSRF guard** for merchant URLs: HTTPS only, resolve DNS and reject private/loopback/link-local ranges.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables**
     - `WebhookEvents` (Id, Provider, ExternalEventId, EventType, RawBody, Signature, ReceivedAt, Status, Attempts, ProcessedAt, LastError) with **unique `(Provider, ExternalEventId)`**.
     - `OutboxMessages` (Id, Type, AggregateId, Payload, CreatedAt, ProcessedAt, Attempts, NextAttemptAt, LockedUntil).
     - `WebhookEndpoints` (Id, MerchantId, Url, EncryptedSecret, EnabledEvents, Status).
     - `WebhookDeliveries` (Id, EndpointId, OutboxMessageId, AttemptNo, ResponseCode, LatencyMs, Status, NextRetryAt).
   - Event catalogue document (`payment.*`, `refund.*`) with versioned schemas.
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] A webhook with a correct signature is stored and processed; with a tampered body or wrong secret it returns 401 and is **not** stored as valid.
   - [ ] A valid signature with a timestamp 10 minutes old returns 400 (replay defence).
   - [ ] The same `eventId` sent 20 times in parallel results in exactly one `WebhookEvents` row and exactly one ledger effect.
   - [ ] `captured` delivered before `authorized`: the final payment state is correct, both events are recorded, and no illegal transition occurs.
   - [ ] The inbound endpoint returns in under 100 ms even if processing takes seconds (processing is asynchronous).
   - [ ] Kill the process after the DB commit but before dispatch: on restart the outbox message is still delivered (zero lost events).
   - [ ] Roll back a payment transaction: **no** outbox row and no merchant notification exists.
   - [ ] A merchant endpoint returning 500 is retried on the documented backoff schedule, then lands in dead-letter status with an alert-able counter.
   - [ ] Two dispatcher instances running concurrently never deliver the same message twice simultaneously (lock test).
   - [ ] Registering `http://169.254.169.254/` or `https://localhost` as a webhook URL is rejected.
   - [ ] Rotating a secret keeps the old one valid for the grace window and both signatures verify at the merchant sample receiver.

Phase 7: Reconciliation Engine & Discrepancy Detection

1. **Domain Logic & Conceptual Explanation:**
   - **What reconciliation is.** Reconciliation answers one question: *"Do all the independent records of this money agree, and can we explain every difference?"* It is the safety net that finds what tests and code reviews miss: a webhook that never arrived, a double capture, a fee charged incorrectly, a refund the provider never executed. Banks and payment companies run it **every day**; unreconciled money is a regulatory and financial emergency.
   - **Three sources of truth.**

Source	What it says	Granularity
FinCore (payments + ledger)	"We captured these payments and owe merchants this much."	Per transaction
Provider settlement report	"These transactions settled; here are gross, fees and net."	Per transaction, daily file
Bank statement	"This lump-sum arrived in your account."	Per batch (net of fees and refunds)

   - **Two levels of matching.**
     - *Level 1 — transaction matching:* FinCore captured payments vs. provider report rows, keyed by our `reference` (provider transaction id as a fallback) plus currency, amount and settlement date.
     - *Level 2 — batch matching:* for each settlement batch, `gross − fees − refunds − chargebacks = net` must equal the bank deposit to the piaster.
   - **Discrepancy taxonomy (learn these names — interviewers love them).**

Type	Meaning	Typical root cause	Severity
`MISSING_AT_PROVIDER`	We say captured; provider has nothing	Capture never reached provider	High
`MISSING_INTERNALLY`	Provider settled something we have no record of	Lost webhook, orphan charge — **customer charged, we know nothing**	Critical
`AMOUNT_MISMATCH`	Same transaction, different amount	Partial capture, bug, FX rounding	High

Type	Meaning	Typical root cause	Severity
`STATUS_MISMATCH`	We say refunded; provider says captured	Refund call failed silently	High
`DUPLICATE`	One transaction appears twice on one side	Duplicate webhook, double capture	Critical
`FEE_MISMATCH`	Provider fee differs from contract	Wrong fee tier	Medium
`TIMING`	Present on both sides but in different days	Cut-off time difference	Low (auto-resolves)
`BATCH_TOTAL_MISMATCH`	Bank deposit differs from report net	Missing row, bank-side fee	Critical

- **Settlement posting.** Once a transaction is *matched*, we post the settlement ledger entry (Dr Bank, Dr Provider Fees, Cr Provider Clearing), flip payments to `Settled`, and move the merchant's balance from **pending to available**. This ties Phases 3–4 to reality.
- **Resolving discrepancies is accounting, not editing.** A resolution (for example *write off a 5-piaster rounding difference* or *book the missing capture*) creates a **new, balanced adjusting journal entry** through `LedgerService`, referencing the discrepancy. History is never altered.

2. **Phase Inputs (Prerequisites):**
   - Phases 3–4 ledger and balances; Phase 5 simulator settlement-report endpoint; Phase 6 events (to explain *why* something is missing).
   - A report format: CSV with columns `provider_txn_id, reference, type (sale|refund|fee|chargeback), amount, fee, net, currency, occurred_at, settled_at`.
   - Anomaly injection in the simulator (configured via scenarios): missing rows, extra rows, altered amounts, duplicated rows, wrong fees.
3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/admin/reconciliation/imports`	Upload or pull a provider report. Idempotent by file SHA-256. Returns `202` + `importId`.

Method	Route	Purpose
GET	`/api/v1/admin/reconciliation/imports/{id}`	Import status, row count, totals, rejected rows.
POST	`/api/v1/admin/reconciliation/bank-statements`	Import bank lines (CSV).
POST	`/api/v1/admin/reconciliation/runs`	Body `{ "provider": "simulator", "date": "2026-10-13" }`; starts a run.
GET	`/api/v1/admin/reconciliation/runs/{id}`	Summary: matched, unmatched, amounts, per-type discrepancy counts.
GET	`/api/v1/admin/reconciliation/discrepancies?status=&type=&severity=`	Work queue for operations.
POST	`/api/v1/admin/reconciliation/discrepancies/{id}/resolve`	Body `{ "resolution": "write_off", "note": "…", "adjustment": { "amount": 5 } }`.
GET	`/api/v1/admin/settlements/{id}`	Settlement batch with matched payments and ledger entry.

```
// GET /api/v1/admin/reconciliation/runs/run_2026_10_13
{ "id": "run_2026_10_13", "status": "completed", "internalCount": 1204,
"providerCount": 1203,
  "matched": 1198, "discrepancies": { "MISSING_AT_PROVIDER": 2,
"AMOUNT_MISMATCH": 1, "MISSING_INTERNALLY": 1, "DUPLICATE": 1 },
  "batch": { "gross": 31250000, "fees": 468750, "refunds": 120000, "net":
30661250, "bankDeposit": 30661250, "matches": true } }
```

Components

- `SettlementReportImporter`: streaming CSV parser, per-row validation, row hash, `SqlBulkCopy` into a staging table.
- `ReconciliationEngine` pipeline: *exact match on reference + amount + currency → tolerant match (±1 day) → classify leftovers → batch total check*. Matching is **set-based SQL joins**, not row-by-row loops.

- `DiscrepancyDetector` (pure, unit-testable classification rules) and `SettlementPoster`.
- `DailyReconciliationScheduler` (hosted service; hardened in Phase 8).

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables:** `ProviderReportFiles` (unique `FileHash`), `ProviderReportRows`, `BankStatementLines`, `ReconciliationRuns`, `ReconciliationMatches` (RunId, PaymentId, ProviderRowId, MatchKind), `Discrepancies` (Id, RunId, Type, Severity, PaymentId?, ProviderRowId?, ExpectedAmount, ActualAmount, Status `Open|Investigating|Resolved|WrittenOff`, ResolvedBy, Note, AdjustmentEntryId), `Settlements` (Id, ProviderDate, Gross, Fees, Refunds, Net, BankLineId, Status).
   - A daily reconciliation report (JSON + CSV) suitable for a finance team.
   - A documented discrepancy runbook (what to do for each type).
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] Clean day: 1,000 payments in both systems produce 1,000 matches, zero discrepancies, batch total ties to the bank line.
   - [ ] Inject exactly 1 missing-at-provider, 1 missing-internally, 1 amount mismatch (25,000 vs 24,900), 1 duplicate and 1 wrong fee: the engine reports **exactly those five**, correctly typed, with no false positives.
   - [ ] A payment captured at 23:59:50 UTC and reported next day is classified `TIMING`, not a failure, and matches on the following run.
   - [ ] Re-running the same date produces identical results and **no duplicate settlement ledger entries** (idempotent runs).
   - [ ] Importing the same file twice (same hash) is rejected or no-ops; a file with one malformed row imports the rest and lists the rejected row with its line number.
   - [ ] Performance: 1,000,000 provider rows reconcile in a bounded time budget (set a target, for example under 2 minutes) with flat memory use.
   - [ ] Batch identity holds: gross − fees − refunds − chargebacks = net = bank deposit; a deliberate 1-piaster difference raises `BATCH_TOTAL_MISMATCH`.
   - [ ] Resolving a discrepancy writes a balanced adjusting entry referencing it; resolving it a second time returns 409.
   - [ ] Settlement posting moves merchant `pending` to `available` by exactly the settled net and the trial balance still nets to zero.
   - [ ] No reconciliation operation executes `UPDATE` or `DELETE` on ledger tables.
   - [ ] Currency mismatch (EGP vs USD with the same numeric amount) is never matched.

Phase 8: Background Workers, Retries & Caching

1. **Domain Logic & Conceptual Explanation:**
   - **Why workers are the heartbeat of a payment system.** Most of what makes payments *reliable* happens off the request path: delivering events, resolving unknown outcomes, expiring stale payments, verifying the ledger, reconciling. Workers must survive crashes, deploys and duplicate instances.
   - **The reliability toolkit.**

Pattern	Problem it solves
At-least-once + idempotent handlers	Crashes cause re-delivery; handlers must tolerate it.
Leasing (`LockedUntil`)	A worker that dies mid-job releases the job automatically when the lease expires.
Exponential backoff + jitter	Retrying instantly hammers a struggling dependency; jitter avoids synchronized retry storms.
Transient vs. permanent errors	Retry timeouts; never retry `400 Bad Request` forever.
Dead-letter queue (DLQ)	A poison message must not block the queue; park it, alert, let humans decide.
Leader election / applock	A scheduled job must run once even with 3 app instances (`sp_getapplock` in SQL Server).
Graceful shutdown	On deploy, finish in-flight work; never abandon a half-posted entry.

   - **Caching: what is safe and what is deadly.** Cache *read-mostly reference data*: API-key-to-merchant lookups, currency exponents, fee schedules, merchant settings. **Never** cache anything used to decide whether money may move (available balance for a refund, payment status in a state transition) — stale data there means overspending. Use **cache-aside** with short TTLs, explicit invalidation on change (revoking an API key must take effect immediately), and **stampede protection** (single-flight per key) so 500 concurrent misses cause one database call.
   - **Real-world problem solved.** Stripe-style platforms stay correct under crashes and partial failures only because every asynchronous path is retried safely and every stuck state has an owner (a sweeper) that eventually resolves it.
2. **Phase Inputs (Prerequisites):**

- Outbox and inbound-webhook processors (Phase 6); provider inquiry (Phase 5); integrity verifier (Phase 4); reconciliation runner (Phase 7); idempotency cleanup (Phase 2).
- Decisions: lease duration (60 s), max attempts (8), backoff base (10 s, factor 3, ±20% jitter), stuck thresholds (Processing > 2 min, Created > 24 h).
- Optional: Redis via `IDistributedCache`; otherwise `IMemoryCache` with an invalidation interface ready to swap.

3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
GET	`/api/v1/admin/jobs`	Each job's schedule, last run, last status, items processed.
POST	`/api/v1/admin/jobs/{name}/trigger`	Manual run (still takes the distributed lock).
GET	`/api/v1/admin/dead-letters?source=`	Parked failures with error text and attempts.
POST	`/api/v1/admin/dead-letters/{id}/requeue`	Retry a parked item after a fix.
GET	`/health/live` and `/health/ready`	Liveness (process up) and readiness (DB, provider circuit, backlog size).
DELETE	`/api/v1/admin/cache/{key}`	Targeted invalidation.

Worker catalogue

Worker	Cadence	Responsibility
`OutboxDispatcher`	1 s poll	Claim with lease, deliver, mark done, back off on failure.
`InboundWebhookProcessor`	1 s poll	Apply stored provider events.
`StuckPaymentSweeper`	1 min	`Processing` / `PendingReview` older than 2 min → provider inquiry; `Created` older than 24 h → `Expired`.
`IdempotencyKeyCleanup`	hourly	Delete expired keys and stale `InProgress` locks.
`LedgerIntegrityJob`	nightly	Hash chain + projection verification; raises critical alert on any mismatch.

Worker	Cadence	Responsibility
`DailyReconciliationScheduler`	daily	Pull report, run reconciliation, publish summary.
`DeadLetterMonitor`	5 min	Exposes DLQ depth as a metric.

Components: a `JobRunner` base class (lease, retry policy, metrics, `JobRuns` logging, cancellation handling); `TimeProvider` injected everywhere for testable time; `ICacheService` with single-flight `GetOrCreateAsync`; cache decorators over merchant and currency repositories.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables:** `JobRuns` (Id, JobName, StartedAt, FinishedAt, Status, ItemsProcessed, Error); `DeadLetters` (Id, Source, MessageId, Payload, LastError, Attempts, FailedAt, Status).
   - All workers running as hosted services (same process locally, separable into `FinCore.Workers` for deployment).
   - A retry/backoff policy document and a "what may be cached" decision table.
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] Kill a worker mid-batch: the lease expires and another worker finishes the work; no message is lost and no side effect is duplicated.
   - [ ] Using a fake `TimeProvider`, failures retry at 10 s, 30 s, 90 s … (within jitter bounds) and the 8th failure dead-letters the message.
   - [ ] A poison message (always throws) goes to `DeadLetters` and the queue behind it keeps flowing.
   - [ ] Three app instances start: each scheduled job executes once per tick (applock proven by `JobRuns` rows).
   - [ ] A transient provider outage causes retries and recovery; a `400`-class permanent failure is **not** retried.
   - [ ] `SIGTERM` during processing: in-flight items finish (or are safely released) within the shutdown timeout; no half-posted journal entries.
   - [ ] The sweeper resolves a payment stuck in `Processing` because the process died between the two transactions of Phase 5.
   - [ ] Revoking an API key takes effect immediately (explicit invalidation) even with a 5-minute TTL.
   - [ ] 500 concurrent cache misses on one key cause exactly **one** database query.
   - [ ] Refund and capture decisions read balances and statuses from the database, never from cache (verified by test with a deliberately stale cache).

- [ ] `/health/ready` turns unhealthy when the database is unreachable or the outbox backlog exceeds the threshold.
- [ ] Requeuing a dead letter after fixing its cause processes it exactly once.

Phase 9: Security & Observability

1. **Domain Logic & Conceptual Explanation:**
   - **Why money systems are prime targets.** Attackers want to read card/PII data, steal merchant balances, forge webhooks, or enumerate other merchants' payments. Regulators (PCI-DSS, central bank cyber rules, GDPR-style privacy laws) require demonstrable controls. Security here is not a feature added at the end — it is a set of **invariants**: *a merchant can only ever see its own data; every sensitive action is attributable to a person; secrets never appear in logs.*
   - **Threat model (lightweight STRIDE).**

Threat	FinCore example	Control
Spoofing	Stolen API key	Hashed keys, scopes, rotation, per-key rate limits, last-used tracking
Tampering	Altered ledger row	Phase 4 hash chain, DENY permissions, audit log
Repudiation	"I never approved that write-off"	Append-only `AuditLogs`, maker-checker
Information disclosure	Merchant A reads Merchant B's payment (**BOLA**, OWASP API #1)	EF **global query filter** by `MerchantId`, 404 on foreign ids
Denial of service	Burst of requests exhausts DB	Rate limiting (429 + `Retry-After`), request size limits, timeouts
Elevation of privilege	Support agent reverses ledger entries	RBAC roles, separation of duties

   - **Maker-checker (four-eyes principle).** Dangerous actions (manual reversal, write-off, payout approval) need one person to *request* and a *different* person to *approve* — a standard banking control you can implement and defend in your viva.
   - **Observability = being able to ask new questions of a running system.** Three pillars:
     - **Structured logs** (Serilog, JSON): `PaymentId`, `MerchantId`, `CorrelationId` on every line; secrets and PAN-like patterns redacted by a destructuring policy.

    - **Metrics** (OpenTelemetry → Prometheus): technical *and business* signals.
    - **Distributed traces**: a request is followed from API → DB → outbox → worker → merchant webhook by persisting the `traceparent` in the outbox row.
  - **Business-level alerts matter more than CPU graphs.** Page someone when *unreconciled amount > 0 for 24 h*, *ledger integrity failure*, *stuck payments > N*, *DLQ depth > 0*, or *authorization success rate drops 10%*.
2. **Phase Inputs (Prerequisites):**
  - All previous phases running; admin endpoints already prefixed `/api/v1/admin`.
  - ASP.NET Core Identity or an OIDC provider (Keycloak/Entra ID) for staff authentication.
  - Seq/Grafana/Prometheus (containers are fine) for local observability; OpenTelemetry SDK packages.
3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
POST	`/api/v1/api-keys`	Create a key with `{ "name", "mode": "test`
POST	`/api/v1/api-keys/{id}/roll`	Issue replacement; old key valid for a grace period.
DELETE	`/api/v1/api-keys/{id}`	Revoke (immediate; cache invalidated).
POST	`/api/v1/admin/auth/token`	Staff login (or OIDC redirect) issuing short-lived JWT with role claims.
GET	`/api/v1/admin/audit-logs?actor=&entity=&from=&to=`	Searchable audit trail.
POST	`/api/v1/admin/approvals`	Create approval request (reversal, write-off).
POST	`/api/v1/admin/approvals/{id}/approve`	Different staff member approves; executes the action.
GET	`/metrics`	Prometheus scrape (internal network only).

Components & middleware

- `SecurityHeadersMiddleware` (HSTS, `X-Content-Type-Options`, no `Server` header); HTTPS redirection; strict CORS.

- ASP.NET Core `RateLimiter`: token bucket per API key, stricter limits on auth endpoints.
- `RequestSizeLimit` and strict JSON parsing (reject unknown fields to prevent mass assignment).
- `MerchantScopeQueryFilter` (EF Core) + a test that fails if any entity with `MerchantId` lacks it.
- `AuditSaveChangesInterceptor` writing who/what/when/before/after to `AuditLogs`.
- `LogRedactionPolicy`; OpenTelemetry instrumentation for ASP.NET Core, HttpClient, SqlClient and custom `ActivitySource`s.
- **Metric catalogue:** `fincore_payments_created_total{status}`, `fincore_payment_transition_total{from,to}`, `fincore_provider_latency_seconds` (histogram), `fincore_outbox_backlog`, `fincore_dead_letters`, `fincore_ledger_integrity_failures_total`, `fincore_unreconciled_amount_minor`, `fincore_stuck_payments`.

4. **Phase Outputs (Tangible Artifacts):**
   - **Tables:** `ApiKeys` (Id, MerchantId, Prefix, KeyHash, Mode, Scopes, Status, LastUsedAt, ExpiresAt, CreatedAt) replacing `Merchants.ApiKeyHash`; `AuditLogs` (Id, ActorType, ActorId, Action, EntityType, EntityId, BeforeJson, AfterJson, CorrelationId, Ip, OccurredAt) insert-only; `ApprovalRequests` (Id, Action, Payload, RequestedBy, ApprovedBy, Status, CreatedAt, DecidedAt).
   - Grafana dashboards (golden signals + business KPIs) and alert rules as code.
   - A one-page threat model and an OWASP API Top 10 checklist mapped to your controls.
5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] **BOLA test:** for every GET/POST that takes an id, calling it with another merchant's id returns 404 (automated across all routes).
   - [ ] A key with only `payments:read` calling `POST /payments` receives 403 `insufficient_scope`.
   - [ ] A revoked key is rejected on the very next request; a rolled key works during grace and fails after it.
   - [ ] 100 requests in one second from one key: the excess receives 429 with `Retry-After`; another merchant is unaffected.
   - [ ] The person who requested a reversal cannot approve it (403); after a different admin approves, exactly one reversal entry exists.
   - [ ] Every state-changing admin action produces one `AuditLogs` row with actor, before/after and correlation id; `UPDATE`/`DELETE` on `AuditLogs` is denied.

- [ ] Search all logs after a test run for API keys, webhook secrets, `Authorization` headers and 13–19 digit numbers: **zero** hits.
- [ ] Unknown JSON fields (for example `"status": "captured"` in create-payment) are rejected with 400/422.
- [ ] A single trace id links the API request, the DB spans, the outbox dispatch and the outgoing merchant webhook.
- [ ] Killing the provider simulator causes the *provider error rate* alert to fire within the configured window; a lost webhook scenario raises the *unreconciled amount* alert after the next run.
- [ ] Dependency scan (`dotnet list package --vulnerable`) reports no High/Critical findings.

Phase 10: Docker, CI/CD & Production Readiness

1. **Domain Logic & Conceptual Explanation:**
   - **"Works on my machine" is unacceptable for money.** Containers make the runtime identical in development, CI and production. A pipeline makes every change *prove* the golden invariants before it can reach customers: no one deploys a build whose ledger tests fail.
   - **Safe deployment of a financial system.**
     - **Database migrations are the riskiest step.** Use the *expand/contract* approach: first deploy backward-compatible schema changes (add column nullable), then code that uses it, later remove the old column. Generate idempotent migration bundles (`dotnet ef migrations bundle`) and run them as a separate pipeline job — never let production instances race to migrate on startup.
     - **Rolling deployments** rely on Phase 8's readiness probes and graceful shutdown so in-flight payments finish.
     - **Backups and recovery objectives:** define *RPO* (how much data you may lose — near zero for a ledger, via full + log backups / point-in-time restore) and *RTO* (how fast you must be back). A backup is only real once a **restore has been tested**.
   - **Testing beyond unit tests.** The test pyramid for FinCore: unit (state machine, money, fees) → integration with real SQL Server (Testcontainers) → concurrency tests → property-based ledger tests → contract tests against the OpenAPI spec → load and chaos tests → a full **"day in the life"** scenario.
   - **Why this phase matters for your graduation project.** Examiners and employers rarely see a student project that can be started with one command, has an automated pipeline, and proves its own financial correctness. This phase is what turns *code* into a *product*.
2. **Phase Inputs (Prerequisites):**

- Phases 1–9 complete with green tests; configuration moved to environment variables / secret stores.
- A Git repository (GitHub), a container registry (GHCR/Docker Hub), and a target environment (a single VM with Docker Compose, Azure Container Apps, or a small Kubernetes cluster).

3. **Endpoints & Component Deliverables:**

Method	Route	Purpose
GET	`/version`	Build info: semantic version, git SHA, build time (no secrets).
GET	`/health/live`, `/health/ready`	Probes used by Docker/Kubernetes and the pipeline smoke test.
GET	`/swagger`	Enabled in non-production only.

Deliverables

- **Dockerfiles** (multi-stage, non-root user, read-only filesystem where possible, `HEALTHCHECK`) for `api`, `workers` and `provider-simulator`.
- **`docker-compose.yml`**: `api`, `workers`, `simulator`, `sqlserver` (with named volume), `seq`, `prometheus`, `grafana`, optional `redis`; a one-shot `migrator` service runs first.
- **CI workflow** (every pull request): restore → build with warnings as errors → format check → unit + integration tests (Testcontainers) → coverage gate (domain ≥ 90%) → vulnerable-package scan → secret scan (gitleaks) → container build → image scan (Trivy).
- **CD workflow** (on merge to main): build and push image tagged with git SHA → apply migration bundle to staging → deploy → smoke tests → k6 load test → **manual approval** → production deploy.
- **Test assets:** k6 scripts (create/confirm/capture/refund mix), a chaos script (kill DB connection, pause simulator, duplicate webhooks), and `DayInTheLifeTests` end-to-end.
- **Documentation:** README quick-start, architecture diagrams (C4 context + container), ADRs for every decision in Part 0, operational runbooks (stuck payment, discrepancy, failed migration, restore).

4. **Phase Outputs (Tangible Artifacts):**

- A repository where `docker compose up` yields a fully working, observable FinCore stack.
- Green pipeline badges, published images, migration bundle artifacts.
- A load-test report (throughput, p50/p95/p99, error rate) and a chaos-test report.

- A backup/restore drill record with the measured RTO.
- A final thesis demo script: onboard merchant → pay → timeout → webhook → refund → reconcile → resolve discrepancy → show trial balance and audit trail.

5. **Measurable Completion Checklist (Definition of Done):**
   - [ ] On a clean machine, `docker compose up` brings every service to healthy status in under 5 minutes with no manual steps beyond copying `.env.example`.
   - [ ] Containers run as non-root; the image contains no secrets (verified by scan) and the repo has no committed credentials.
   - [ ] A deliberately failing test, a coverage drop below the gate, or a High-severity vulnerable package **blocks** the pipeline.
   - [ ] The migration bundle is idempotent: running it twice is a no-op; running it against the previous release's schema keeps the old app version working (expand/contract proven).
   - [ ] Rolling deploy during a k6 load of 100 requests/s produces **zero** failed payments and zero lost or duplicated ledger entries.
   - [ ] Load target met (set your own, for example 200 payment creations/s, p95 under 300 ms) while integrity checks show **zero** invariant violations afterwards.
   - [ ] Chaos run (DB blip, provider outage, 30% duplicate webhooks, restarts) finishes with every payment in a legal state, trial balance zero, integrity job green and reconciliation reporting only the intentionally injected anomalies.
   - [ ] Restore of last night's backup to a fresh SQL Server instance passes `/admin/ledger/integrity`, and the measured restore time is recorded against the RTO.
   - [ ] The *Day in the Life* test (1,000 payments, 10% refunds, 5% declines, 2% timeouts, daily reconciliation) ends with assertions: total debits = total credits, merchant balances = sum of settled nets, zero unexplained discrepancies.
   - [ ] `/version` reports the exact git SHA that the pipeline deployed.

Appendix — How to Work Through This Roadmap

A. Suggested Timeline (Part-Time Student Pace)

Weeks	Phases	Milestone you can demo
1–2	1–2	Safe, idempotent payment API with a legal state machine

Weeks	Phases	Milestone you can demo
3–5	3–4	A ledger that balances and cannot be rewritten; refunds
6–8	5–6	Provider simulator, unknown outcomes, webhooks and outbox
9–11	7	Reconciliation catching injected discrepancies
12–13	8	Workers that survive kills; caching rules
14–16	9–10	Hardened, observable, containerised, pipeline-verified system

B. Cumulative Data Model (Where Each Table Is Born)

Phase	Tables introduced
1	`Merchants`, `Customers`, `Payments`
2	`IdempotencyKeys`, `PaymentStatusHistory`
3	`LedgerAccounts`, `JournalEntries`, `JournalLines`
4	`AccountBalances`, `Refunds` (+ hash columns)
5	`PaymentAttempts`, `ProviderCallLogs`
6	`WebhookEvents`, `OutboxMessages`, `WebhookEndpoints`, `WebhookDeliveries`
7	`ProviderReportFiles`, `ProviderReportRows`, `BankStatementLines`, `ReconciliationRuns`, `ReconciliationMatches`, `Discrepancies`, `Settlements`
8	`JobRuns`, `DeadLetters`
9	`ApiKeys`, `AuditLogs`, `ApprovalRequests`

C. Recommended Libraries

- **Core:** EF Core (SQL Server), FluentValidation, MediatR or plain application services, Polly (`Microsoft.Extensions.Resilience`).
- **Testing:** xUnit, FluentAssertions, Testcontainers, FsCheck (property-based), WireMock.Net, k6.

- **Operations:** Serilog + Seq, OpenTelemetry, Prometheus + Grafana, HealthChecks, Quartz.NET (optional scheduler).

D. Rules That Keep You Sane

1. **Finish each phase's checklist before starting the next.** Phases are designed so every later phase leans on the guarantees of the earlier ones.
2. **Write the failing test first for every invariant** (balanced entries, legal transitions, exactly-once effects). Concurrency bugs are cheap to find in tests and ruinous in production.
3. **When in doubt, record a fact and decide later.** Raw webhooks, provider calls and ledger entries are stored before they are interpreted.
4. **Keep a design diary.** For your graduation defence, the *reasons* behind each decision (ADRs) are worth as much as the code.
5. **Be able to explain one journal entry end to end.** If you can trace 250.00 EGP from the customer's tap, through the provider, into each ledger account, into settlement and onto the bank statement, you understand FinTech.